

Basic Building Blocks

Ex: Generics

Course: Rust Basics

Author: Aleksandr Kugushev

Generics

```
struct MyPair<T> {  
    left: T,  
    right: T,  
}
```

```
let my_small = MyPair::<i8> { left: 42, right: 1 };  
let my_big = MyPair::<i128> {  
    left: 42424212123123123123123123312312312,  
    right: 4121234236423648726348263874621,  
};
```

Generic Functions

```
fn generic_function<T>(p1: T, p2: T) -> MyPair<T> {  
    MyPair::<T>{  
        left: p1,  
        right: p2  
    }  
}  
  
generic_function::<i32>(42, 1);  
generic_function(42, 1);
```

Multiple Generics

```
fn generic_function<T, V>(p1: T, p2: T, other: V) -> MyPair<T> {  
    MyPair::<T>{  
        left: p1,  
        right: p2  
    }  
}  
  
generic_function::<i32, bool>(42, 1, true);  
generic_function(42, 1, true);
```

Generic Methods

```
struct MyPair<T> {  
    left: T,  
    right: T,  
}  
impl MyPair<i32> {  
    fn do_something(&self) {println!("{}", self.left + self.right) }  
}
```

```
let pair_of_i32 = MyPair::<i32> { left: 42, right: 1 };
```

```
pair_of_i32.do_something();
```

```
let pair_of_i8 = MyPair::<i8> { left: 42, right: 1 };
```

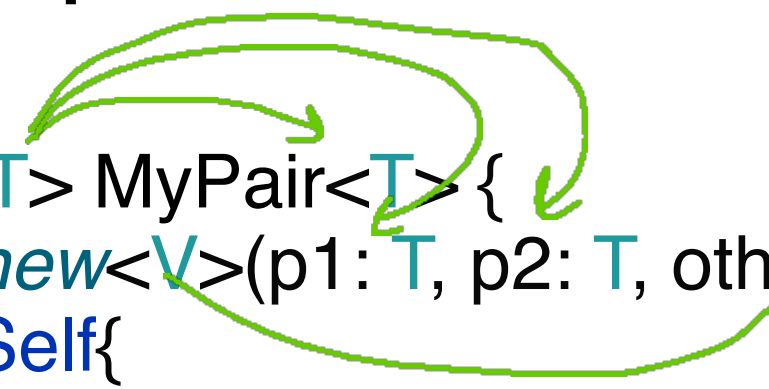
```
pair_of_i8.do_something(), method not found in `MyPair<i8>`
```

Generic impl

```
impl<T> MyPair<T> {  
    fn do_something_with_type(&self) {  
        println!("{}", self.left + self.right)  
    }  
}
```

Multiple Generics in Impl

```
impl<T> MyPair<T> {  
    fn new<V>(p1: T, p2: T, other: V) -> Self {  
        Self{  
            left: p1,  
            right: p2  
        }  
    }  
}
```

A diagram with three green curved arrows illustrating generic parameter resolution. The first arrow starts at the first <T> in the impl signature and points to the T parameter in the MyPair struct definition. The second arrow starts at the second <T> in the MyPair struct definition and points to the T parameter in the new function signature. The third arrow starts at the <V> in the new function signature and points to the V parameter in the Self struct definition.

```
let pair = MyPair::<i32>::new::<bool>(42, 1, true);  
let pair = MyPair::new(42, 1, true);
```

Generic Restrictions

```
impl<T> MyPair<T> {  
    fn do_something_with_type(&self) {  
        println!("{}", self.left + self.right) error[E0369]: cannot add `T` to `T`  
    }  
}
```

help: consider restricting type parameter `T`

```
impl<T: std::ops::Add> MyPair<T> {
```


Generic Bounds

```
impl<T: Add> MyPair<T> {  
    fn do_summ(self) {  
        let sum = self.left + self.right;  
    }  
}
```

Generic Bounds

```
trait ToString {  
    fn to_string(&self) -> String;  
}
```

```
impl<T: ToString> MyPair<T> {  
    fn print(&self) {  
        println!("{}", self.left.to_string(), self.right.to_string())  
    }  
}
```

Trait Display

```
impl<T: Display> MyPair<T> {  
    fn print(&self) {  
        println!("{}", self.left, self.right)  
    }  
}  
  
pub trait Display {  
    #[stable(feature = "rust1", since = "1.0.0")]  
    fn fmt(&self, f: &mut Formatter<'_>) -> Result;  
}
```

Impl Display

```
impl<T: Display> Display for MyPair<T> {  
    fn fmt(&self, f: &mut Formatter<'_>) -> std::fmt::Result {  
        write!(f, "({}, {})", self.left, self.right)  
    }  
}
```

```
let pair = MyPair { left: 42, right: 1 };  
println!("{}", pair);
```

Multiple Bounds

```
fn trace<T: Display>(to_trace: T, verbose: bool) {  
    println!("Display: `{}`", to_trace);  
    if verbose {  
        println!("Debug: `{:?}`", to_trace);  
    }  
}
```

error[E0277]: `T` doesn't implement `Debug`

Multiple Bounds

```
fn trace<T: Display + Debug>(to_trace: T, verbose: bool) {  
    println!("Display: {}", to_trace);  
    if verbose {  
        println!("Debug: `{:?}`", to_trace);  
    }  
}
```

```
#[derive(Debug)]
```

```
struct MyPair<T> { left: T, right: T, }
```

```
> Display: `(42, 1)`
```

```
> Debug: `MyPair { left: 42, right: 1 }`
```

Where clause

```
fn trace_multiple<T: Display + Debug, V: Display +  
Debug>(to_trace1: T, to_trace2: V) {...
```

```
fn trace_multiple<T, V>(to_trace1: T, to_trace2: V)  
  where  
    T: Display + Debug,  
    V: Display + Debug {
```

Trait Sized

```
trait Power {  
    fn amount(self) -> i32;  
    fn is_powerful(self) -> bool { (self) <- doesn't have a size known at compile-time  
        self.amount() > 9000  
    }  
}
```

error[E0277]: the size for values of type `Self` cannot be known at compilation time

Trait Sized

Types with a constant size known at compile time.
All type parameters have an implicit bound of `Sized`

```
trait Power {  
    fn amount(self) -> i32;  
    fn is_powerful(self) -> bool {  
        self.amount() > 9000  
    }  
}
```

“I can’t guarantee that `Self` is sized, so I can’t take it as a parameter”

DST (Dynamically Sized Types)

```
fn is_powerful(value: [u32]) -> bool {  
    todo!()  
}
```

error[E0277]: the size for values of type `[u32]` cannot be known at compilation time

Trait Sized

```
trait Power {  
    fn amount(self) -> i32;  
    fn is_powerful(self) -> bool where Self: Sized {  
        self.amount() > 9000  
    }  
}
```

Associated Type

```
trait FactoryAssociated{  
    type Item;  
    fn create_a() -> Self::Item;  
}  
  
trait FactoryGeneric<T>{  
    fn create_g() -> T;  
}
```

Associated Type

```
impl FactoryGeneric<i32> for  
MyFactory {  
    fn create_g() -> i32 {  
        42  
    }  
}
```

```
impl FactoryAssociated for  
MyFactory {  
    type Item = i32;  
  
    fn create_a() -> Self::Item {  
        42  
    }  
}
```

Associated Type

```
impl FactoryGeneric<i32> for  
MyFactory {  
    fn create_g(self) -> i32 { 42 }  
}
```

```
impl FactoryGeneric<bool> for  
MyFactory {  
    fn create_g(self) -> bool  
    { true }  
}
```

Associated Type

```
impl FactoryAssociated for  
MyFactory {  
    type Item = i32;  
    fn create_a(self) -> Self::Item  
    { 42 }  
}
```

```
impl FactoryAssociated for  
MyFactory {  
    type Item = bool;  
    fn create_a(self) -> Self::Item  
    { true }  
}
```

error[E0119]: conflicting implementations of trait
`FactoryAssociated` for type `MyFactory`

Associated Type defines PAIR

```
let my_factory = MyFactory;
```

```
let val = my_factory.create_a();
```

```
let val = my_factory.create_g();
```

```
error[E0282]: type annotations needed
```


Associated Type defines PAIR

```
let my_factory = MyFactory;
```

```
let val = my_factory.create_a();
```

```
let val: i32 = my_factory.create_g();
```

Why do we need tuples?

- Multiple return
- Traits
- Generic

Generic Tuples

```
fn my_trace<T: Debug>(item: T) {  
    println!("{:?}", item);  
}
```

```
let tuple = (42, "hello", true);  
my_trace(tuple);
```

```
> (42, "hello", true)
```